

Python Concurrency Patterns

Python provides three main approaches to concurrency: threading, multiprocessing, and asyncio. Each serves different use cases depending on whether the workload is I/O-bound or CPU-bound.

Threading (I/O-Bound)

Python's threading module uses OS-level threads but is limited by the Global Interpreter Lock (GIL). Threads share memory space, making them suitable for I/O-bound tasks like network requests, file operations, and database queries. The `concurrent.futures.ThreadPoolExecutor` provides a high-level interface for managing thread pools.

Multiprocessing (CPU-Bound)

For CPU-bound workloads, multiprocessing bypasses the GIL by running separate Python processes. Each process has its own memory space and interpreter. `ProcessPoolExecutor` simplifies parallel execution. Inter-process communication uses `Queue`, `Pipe`, or shared memory (`multiprocessing.Value`, `Array`). The overhead of process creation makes this best for coarse-grained parallelism.

Asyncio (Cooperative Multitasking)

Asyncio provides cooperative multitasking using coroutines (`async/await`). A single event loop schedules coroutines, switching between them at `await` points. This is ideal for high-concurrency I/O workloads like web servers (`FastAPI`, `aiohttp`) and database drivers (`asyncpg`, `motor`). Unlike threading, asyncio avoids race conditions since only one coroutine runs at a time.

Comparison Table

Approach	Best For	GIL Impact	Overhead
Threading	I/O-bound	Limited	Low
Multiprocessing	CPU-bound	Bypassed	High
Asyncio	High-concurrency I/O	N/A (single thread)	Very Low